DB에서 일기 원문을 숨기기까지

AES-GCM과 HMAC을 선택하고, 저장부터 재시도까지 연결한 과정

아이큐(한동희) | 레벨4 테크니컬 라이팅 초안 | 2026-11-01

대상 독자: Java로 서버를 개발하며, 개인정보를 DB에 저장할 때 암호화를 어디까지 적용해야 할지 고민하는 개발자. Java와 HTTP 요청 처리에 대한 기초 지식을 전제로 한다. 암호학 용어는 필요한 곳에서 설명한다.

1. 암호화보다 먼저 정한 것은 보호할 범위였다

하루들은 사용자의 일기를 네 컷 만화로 만드는 서비스다. 사용자가 입력한 원문은 DB에 저장하고, 본인 상세 조회와 서버의 AI 생성에 다시 사용한다. 입력은 최대 300 Unicode code point로 제한한다. 여기서 code point는 유니코드 문자를 세는 단위다. 화면에서 한 글자로 보이는 이모지가 여러 code point로 구성될 수도 있다.

원문을 저장해야 기능을 제공할 수 있지만, DB 조회 권한이 있으면 사적인 내용을 그대로 읽을 수 있다는 문제가 있었다. 이 문제를 해결하려면 암호 알고리즘을 고르기 전에 **어떤 권한을 가진 사람에게서 무엇을 보호할지** 정해야 했다.

이번 목표는 다음과 같다. **애플리케이션의 비밀 키 없이 DB 조회 결과나 논리 덤프만 얻은 사람이 신규 일기 원문을 직접 읽지 못하게 한다.** 논리 덤프는 SQL 조회 등으로 행과 값을 파일로 내보낸 결과다. 디스크를 암호화하더라도 정상 조회로 내보낸 값에 원문이 있다면 이 목표를 충족하지 못한다.

```
사용자 입력
  -> 애플리케이션: 원문 검증, 비밀 키로 암호화
  -> DB: 암호문 저장

본인 조회
  -> 애플리케이션: 소유권 확인, 키로 복호화
  -> 사용자에게 원문 응답
```

이 경계에서는 서버가 원문을 읽을 수 있어야 한다. 서버에서 원문을 AI에 전달해야 하고, 본인에게 다시 보여 주어야 하기 때문이다. 따라서 비밀번호처럼 복원할 수 없는 형태로만 저장하는 방법은 맞지 않는다. 서버조차 원문을 볼 수 없게 하는 End-to-End Encryption(종단간 암호화)을 적용하려면 AI 호출과 키 복구 등 제품의 동작부터 다시 설계해야 한다.

또한 DB 암호화만으로 일기 전체의 비밀이 보장되는 것은 아니다. 제목, 스토리보드, 생성 이미지에는 원문에서 파생된 내용이 담길 수 있다. 애플리케이션 프로세스와 키가 함께 탈취되거나, 다른 사용자의 조회를 허용하는 권한 오류가 생기면 별도 대응이 필요하다. 이 글의 대상은 **저장된 원문과 요청 지문을 보호하는 설계**다.

설계의 출발점에는 세 가지 조건이 있었다. 원문을 복원할 수 있어야 한다. 암호문이 손상되거나 다른 일기로 옮겨지면 거절해야 한다. 기존 Java 21과 JPA, PostgreSQL 흐름에 통합할 수 있어야 한다. 원문 검색이나 DB 문자열 연산 요구는 없어, 원문을 암호문으로 바꾸더라도 해당 기능을 유지할 부담은 없었다.

이 글은 신규 저장 암호화 PR #442와 기존 데이터 전환 후속 PR #450의 코드를 바탕으로 한다. 코드 설명은 후속 작업의 커밋 7f54ca12를 기준으로 한다. 구현한 내용과 실제 운영 데이터 전환을 완료했다는 주장은 구분한다.

2. Base64, AES-GCM, KMS는 서로 다른 질문의 답이었다

검토 과정에서 Base64로 저장하면 충분한지도 살펴봤다. **Base64는 바이트를 문자열로 표현하는 Encoding(인코딩) 규칙이다.** 비밀 키 없이 누구나 되돌릴 수 있다. 원문에 Base64만 적용하면 DB 유출 시 원문을 숨길 수 없다. 반면 암호문 바이트를 TEXT 열에 저장할 때는 사용할 수 있다. RFC 4648의 보안 고려사항도 인코딩에 기밀성을 기대해서는 안 된다고 설명한다.

보호를 어느 계층에 둘지도 비교했다. 디스크 암호화는 저장 매체를 보호하는 별도 계층이다. PostgreSQL의 pgcrypto는 SQL 안에서 암복호화할 수 있지만, 키와 평문이 DB 서버의 처리 경계를 통과한다. DB 밖에 키를 두려는 이번 목표에는 **애플리케이션에서 암호화한 결과만 DB에 전달하는 방식**이 맞았다. 이는 pgcrypto가 나쁜 선택이라는 뜻이 아니라 신뢰할 대상을 어디까지 둘지의 차이다.

PostgreSQL 보안 제한 문서

그다음 비교한 것은 암호 알고리즘의 사용 방식이다. AES는 같은 비밀 키로 암호화와 복호화를 수행하는 대칭키 알고리즘이다. AES-256의 256은 키 길이로, 32바이트를 뜻한다. Galois/Counter Mode(GCM)는 AES로 원문을 숨기면서 변조 여부도 검증하는 운용 방식이다.

> 보호 위치 결정: 애플리케이션 필드 암호화
> 암호 방식 결정: AES-GCM
> 키 관리 결정: 환경별 키 목록과 활성 키
> 저장 표현 결정: 키 ID + Base64로 표현한 암호문 묶음

AES-CBC만 사용하면 암호문 변조를 검증할 수 없다. CBC에 HMAC을 결합하면 요구를 충족할 수 있지만, 별도 키와 패딩, 인증 범위, 인증 후 복호화 순서를 올바르게 조합해야 한다. 기존 CBC 데이터와 호환해야 하는 조건도 없었다.

AES-GCM과 ChaCha20-Poly1305는 모두 Authenticated Encryption with Associated Data(AEAD, 추가 데이터 인증을 지원하는 인증 암호화) 방식이다. 두 방식 모두 기밀성과 변조 검증 요구를 충족하고 Java 21에서 사용할 수 있다. 이번에는 JDK의 Cipher에서 암호화와 인증을 함께 처리하고, 일기 식별 정보를 인증 대상에 포함할 수 있는 AES-GCM을 저장 형식 v1으로 채택했다. 직접 조합할 항목을 줄인다는 점이 CBC와 HMAC의 조합보다 유리했다. Java Cipher 문서

다만 이 조건이 ChaCha20-Poly1305보다 AES-GCM이 우수하다는 증명은 아니다. 두 후보가 요구를 충족한 상태에서 하나의 형식을 고정한 결정이다. 실제 서버에서 성능을 비교하지 않았으므로 하드웨어 가속이나 응답 시간 우위를 근거로 삼지 않았다. AES-128도 가능한 선택이다. AES-256은 32바이트 키 규격을 사용하고 키 탐색에 대한 여유를 더 두는 선택이며, 키 유출이나 잘못된 사용법을 해결하지는 않는다.

Key Management Service(KMS, 키 관리 서비스)는 이 알고리즘들과 다른 계층의 선택이다. 접근 제어와 감사, 키 보관을 개선할 수 있지만, 권한과 네트워크, 장애 대응을 운영에 추가한다. 이번에는 앱이 환경별 키를 보관하는 범위로 시작하고 그 책임을 문서화했다. KMS를 사용해도 AES-GCM과 함께 구성할 수 있다.

3. 암호화 결과에는 원문 이외의 정보도 필요했다

AES-GCM에 원문과 키만 넣는 것으로 저장 형식이 완성되지는 않는다. 복호화할 때 같은 조건을 다시 구성할 정보도 필요하다.

nonce와 태그의 역할

먼저 Nonce(논스)는 각 암호화에 사용하는 값이다. GCM에서는 **같은 키로 암호화할 때 nonce를 재사용하면 안 된다.** 비밀 값은 아니므로 암호문과 함께 저장한다. 구현에서는 SecureRandom으로 매번 12바이트를 생성한다. 난수 생성이 중복 가능성을 수학적으로 없애는 것은 아니다. 키별 사용량과 교체 정책도 함께 관리해야 한다. 이 제약은 AES-GCM을 선택하며 감수한 핵심 조건이다. NIST SP 800-38D
다음은 Authentication Tag(인증 태그)다. 복호화할 때 키와 입력 데이터가 암호화 시점의 조건에 맞는지 검증하는 값이다. 이 구현에서는 128비트, 즉 16바이트를 사용한다. AES 키의 256비트와 태그의 128비트는 별도 설정이다.
암호화의 핵심 코드는 다음과 같다. 예외 처리와 임시 배열 정리는 생략한 발췌다.

```
byte[] nonce = new byte[12];
secureRandom.nextBytes(nonce);
Cipher cipher = newCipher(
        Cipher.ENCRYPT_MODE, activeKeyId, nonce, diaryId, userId);
byte[] ciphertext = cipher.doFinal(plaintextBytes);
byte[] payload = ByteBuffer.allocate(nonce.length + ciphertext.length)
        .put(nonce).put(ciphertext).array();
return activeKeyId + ":" + Base64.getEncoder().encodeToString(payload);
```

plaintextBytes는 빈값과 길이를 검사하고 앞뒤 공백을 제거한 원문의 UTF-8 바이트다. 잘못된 유니코드 입력은 대체 문자로 바꾸지 않고 실패시킨다. newCipher()는 키와 nonce, 인증에 필요한 추가 데이터를 설정한다.

실제로 저장하는 바이트

여기서 변수 이름만 보면 놓치기 쉬운 부분이 있다. ciphertext에는 암호문만 있는 것이 아니다. **Java GCM의 doFinal()은 암호문 뒤에 인증 태그를 붙여 반환한다.** 따라서 저장 직전에 태그를 한 번 더 만들거나 붙일 필요가 없다. Java Cipher 문서

```
저장 문자열 = keyId:Base64(payload)
payload    = nonce 12바이트 || 암호문 || 태그 16바이트
||         = 앞뒤 바이트를 순서대로 이어 붙인다는 표기
```

키 ID는 실제 비밀 키가 아니라 서버의 키 목록에서 사용할 키를 찾는 이름이다. 실제 비밀 키는 DB에 넣지 않고, 암호문 앞에 키 ID를 기록한다. 별도의 source_text_version 열에는 저장 형식 버전 1을 기록한다. 버전은 값을 해석하는 규칙이고, 키 ID는 해당 규칙에서 사용할 키를 가리킨다.
예를 들어 원문이 오늘이면 UTF-8로 6바이트다. GCM 암호문의 길이는 이 원문 바이트 길이와 같다. payload는 12 + 6 + 16 = 34바이트이고, Base64로는 48문자다. 키 ID가 k1이면 k1:까지 포함해 51문자를 저장한다. 최대 입력은 UTF-8로 1,200바이트이므로 payload의 상한은 1,228바이트다. 길이 검사는 이 저장 형식을 기준으로 한다.

4. 복호화는 태그 검증과 소유권 확인을 함께 필요로 했다

저장 형식이 정상이라는 사실과 요청자가 읽을 권한이 있다는 사실은 다르다. 본인 상세 조회에서는 DB에서 일기를 가져온 뒤, 요청자와 소유자가 같은지 먼저 확인한다. 그다음 복호화한다. 암호화 클래스의 validateIdentifiers()는 ID가 null인지 검사할 뿐 소유권을 확인하지 않는다.
암호문 자체도 해당 일기에 속하는지 확인해야 한다. A라는 일기의 저장 문자열을 B라는 일기의 원문 열에 복사했다고 가정하자. 키와 nonce, 암호문, 태그가 모두 함께 옮겨지므로, 내용에 대한 인증만으로는 저장 위치가 바뀌었다는 사실을 구분하지 못한다.

암호문을 일기와 연결하는 AAD

이를 위해 Additional Authenticated Data(AAD, 추가 인증 데이터)에 일기와 작성자의 식별 정보를 넣는다. AAD는 숨기지 않지만 암호문과 함께 검증할 정보다. 구현에서는 아래 문자열을 UTF-8로 전달한다.

```
harudle.diary.source-text:v1:<keyId>:<diaryId>:<userId>
```

B 일기를 복호화할 때는 B의 일기 ID와 소유자 ID로 AAD를 만든다. A 일기의 암호문을 만들 때 사용한 AAD와 달라져 태그 검증이 실패한다. 이는 DB가 수정되는 순간 경보를 보내는 기능이 아니다. **해당 값을 복호화하는 시점에 잘못된 연결을 거절하는 기능**이다.

```
일기 조회 -> 소유권 확인 -> 버전과 저장 형식 검사
        -> 저장된 키 ID로 키 선택
        -> 같은 AAD 구성 -> GCM 태그 검증과 복호화
        -> UTF-8 및 일기 규칙 검사 -> 응답
```

복호화에서는 Base64를 해제한 payload의 앞 12바이트를 nonce로 꺼낸다. 나머지는 암호문과 태그다. 실제 호출은 다음과 같다.

```
byte[] nonce = Arrays.copyOfRange(payload, 0, 12);
Cipher cipher = newCipher(
        Cipher.DECRYPT_MODE, envelope.keyId(), nonce, diaryId, userId);
plaintextBytes = cipher.doFinal(payload, 12, payload.length - 12);
return Diary.normalizeSourceText(decodeStrictly(plaintextBytes));
```

doFinal()의 세 번째 인자는 종료 인덱스가 아니라 처리할 바이트 수다. 앞서 본 34바이트 payload라면 시작 위치 12에서 22바이트를 처리한다. 태그 검증에 성공하면 태그를 제외한 원문 6바이트를 반환한다. nonce를 잘라낸다는 것은 입력 범위를 정한다는 뜻이며, 원래 payload에서 앞부분을 삭제하는 연산은 아니다.

태그 검증 뒤에도 남는 검사

이후에도 검사가 남는다. 유효한 태그는 유효한 UTF-8 문자열이나 300 code point 이하의 일기라는 뜻이 아니다. 엄격한 UTF-8 디코딩과 일기 규칙 검사를 다시 수행한다. 형식 검사에서도 알 수 없는 키 ID, 허용 길이 초과, Base64를 다시 인코딩했을 때 표현이 달라지는 입력을 거절한다. 형식 검사만 통과했다고 태그 검증까지 끝난 것은 아니다.
실패 시에는 민감값과 원래 예외를 담지 않는 고정 예외를 사용한다. finally에서 임시 원문 바이트를 0으로 덮지만, Java의 원본 String이나 반환 문자열, 암호 구현 내부의 복사본까지 전부 지웠다는 의미는 아니다. 같은 행의 과거 암호문 재사용이나 AAD에 없는 날짜 변경도 이 설계가 검출하는 대상은 아니다.

5. 원문을 암호화해도 SHA 지문으로 후보를 확인할 수 있었다

원문 열뿐 아니라 재시도를 판별하는 요청 지문도 함께 살펴봐야 했다. 이 지문 역시 원문에 의존하고 있었기 때문이다.

Idempotency(멱등성)는 요청을 반복해도 의도한 효과가 중복되지 않도록 하는 성질이다. 하루들에서는 클라이언트가 보내는 멱등성 키로 기존 요청을 찾고, 요청 지문으로 내용도 같은지 확인한다. 같은 키로 다른 일기를 보내면 재시도로 처리해서는 안 된다.

기존 지문은 사용자 ID와 날짜, 원문 등을 정해진 순서로 연결해 SHA-256으로 계산한 값이었다.

SHA-256은 입력을 32바이트의 해시값으로 바꾸며 같은 입력에는 같은 결과를 만든다. 이를 16진수로 표현하면 64자리가 된다. 실제 연결 규칙은 다음과 같다.

```
v1\n<userId>\n<diaryDate>\n<normalizedDiaryText>
\n은 실제 줄바꿈을 뜻한다.
```

해시에서 임의의 원문을 복원하는 일은 어렵다. 하지만 **예상한 후보가 맞는지 확인하는 일은 가능하다.** 공격자가 DB에서 사용자 ID와 날짜를 읽고, 원문 후보를 오늘도 피곤했다로 정하면 같은 규칙으로 SHA를 계산할 수 있다. 저장 지문과 일치하는지 대조하는 데 비밀 값이 필요하지 않다. 짧고 흔한 문장일수록 이런 후보 대입을 고려해야 한다. 해시 입력에 더하는 공개 값인 salt를 사용해도 공격자가 그 값을 함께 읽어 계산할 수 있어 이 경로는 남는다.

키가 있어야 계산할 수 있는 지문

이를 막기 위해 신규 지문에는 Hash-based Message Authentication Code(HMAC, 비밀 키와 해시로 계산하는 메시지 인증 코드)를 적용했다. 후보의 SHA를 알아도 키 없이는 저장된 HMAC과 대조할 값을 계산할 수 없게 한다. 이 목표는 DB만 유출되고 키나 HMAC 계산 수단은 유출되지 않았다는 조건을 전제로 한다. HMAC 규격 RFC 2104

구현에서는 설정된 AES 키를 루트 키로 삼아 HMAC을 두 번 계산한다. 아래는 실제 입력을 정리한 수식이다.

```
지문용 키 = HMAC-SHA256(루트 AES 키, 키 파생 목적 문자열)
최종 지문 = HMAC-SHA256(지문용 키, 요청 문맥 || SHA의 hex64 문자열)
```

첫 번째 계산의 목적 문자열은 harudle.diary.request-fingerprint.key:v1이다. 같은 루트 키에서는 같은 32바이트 지문용 키가 나온다. 새 난수 키를 만드는 과정이 아니다. 두 번째 계산의 요청 문맥은 아래와 같으며 마지막 콜론도 포함한다.

```
harudle.diary.request-fingerprint:v1:<keyId>:<diaryId>:<userId>:
```

그 뒤에 SHA의 소문자 hex64 문자열을 ASCII 64바이트로 이어 넣는다. SHA를 다시 디코딩한 32바이트가 아니다. 최종 HMAC 32바이트를 소문자 hex64로 저장하고, 중간 SHA를 신규 지문 열에 따로 저장하지 않는다. 기존 CHAR(64) 형식을 유지하지만 값이 만들어지는 계약은 달라진다.

암호화에 쓰는 원시 키를 지문 계산에 그대로 재사용하는 대신, 고정된 목적을 넣어 지문용 키를 분리했다. 별도 무작위 키를 관리하면 권한을 더 세밀하게 나눌 수 있지만 두 키 목록의 교체와 복구 관계를 관리해야 한다. 같은 앱에서 두 작업을 수행하는 현재는 파생 방식을 선택했다. 다만 루트 키가 유출되면 지문용 키도 다시 계산할 수 있다. 목적 분리는 루트 키 유출에 대한 독립적인 방어가 아니다.

6. 재시도와 키 교체가 같은 기준을 사용하게 했다

암호화에 사용하는 nonce는 매번 바뀐다. 따라서 같은 원문을 암호화한 문자열 두 개를 비교해서 동일 요청인지 판단할 수 없다. HMAC 지문은 같은 일기와 사용자, 키 ID와 키 값, 요청 내용이면 같은 결과가 나오므로 재시도 비교에 사용할 수 있다. 이 지문에는 nonce나 암호문 바이트를 넣지 않는다.
새 요청에서는 DiaryCreationClaimService가 원문을 암호화한 일기를 저장한다. 이어서 요청의 SHA를 만들고, 저장된 일기의 키 ID를 사용해 HMAC으로 보호한다. 생성 이력에는 이 최종 지문을 저장한다. 일기와 생성 이력을 함께 처리해, 한쪽만 저장된 상태를 남기지 않는다.

```
처음 요청
  -> 원문 검증과 암호화 -> 일기 저장
  -> SHA 계산 -> HMAC 보호 -> 생성 이력 저장

같은 멱등성 키로 재요청
  -> 기존 이력과 일기 조회 -> 소유권 확인
  -> 입력으로 HMAC 재계산 -> 저장 지문과 비교
  -> 같으면 기존 상태 처리, 다르면 충돌로 거절
```

지문을 계산하는 함수 자체는 원문을 복호화하지 않는다. 저장 문자열의 형식과 키 ID를 읽어 HMAC을 계산한다. 다만 전체 재요청 흐름에는 지문 비교 이후 toClaim()에서 원문을 복호화하는 단계가 있다. “재시도에서는 복호화하지 않는다”라고 전체 흐름을 설명하면 틀리게 된다.
생성 실행 직전에는 ClaimedDiaryGenerationService가 요청의 보호된 지문을 다시 계산한다. 일기 ID와 멱등성 키, 지문, 진행 중을 뜻하는 생성 상태 PROCESSING이 저장된 실행 조건과 맞는지 확인한 뒤 AI를 호출한다. 이 검사는 잘못된 실행 요청을 거절하는 조건이다. 이것만으로 동시 요청에서 외부 호출이 반드시 한 번만 일어난다고 보장하지는 않는다. 잠금과 트랜잭션, 상태 전이는 별도로 설계해야 한다.

새 키로 쓰고, 저장된 키로 읽기

키 교체에서도 같은 기준을 유지한다. 설정에는 여러 키를 보관할 수 있고, 활성 키는 새 암호화에 사용할 키를 뜻한다. 예를 들어 활성 키를 k1에서 k2로 바꿔도 기존 k1: 데이터는 k1으로 읽어야 한다. 재시도 지문 역시 **현재 활성 키가 아니라 저장 문자열에 기록된 키 ID**로 계산해야 기존 값과 비교할 수 있다.

```
키 목록: k1, k2 / 활성 키: k2
기존 k1: 데이터 -> k1으로 복호화 및 지문 재계산
신규 작성       -> k2로 암호화 및 지문 생성
```

활성 키 변경은 기존 행의 재암호화를 뜻하지 않는다. 현재 앱은 메모리에 키 목록을 보관하므로 호스트의 설정 파일만 바꾼다고 갱신되지도 않는다. 키 등록과 검증, 재배포 순서가 필요하다. 구키 제거 전에는 현재 DB뿐 아니라 보존 중인 DB 백업의 복원에도 그 키가 필요한지 확인해야 한다.
기존 행을 새 키로 다시 암호화하면 연결된 HMAC도 바뀌므로 함께 갱신해야 한다. 관련 변경을 모두 반영하거나 모두 취소하는 처리 단위인 트랜잭션으로 묶는 것 외에도 진행 중인 생성과 전환이 겹치지 않게 해야 한다. 키가 유효한지 확인하는 절차와 예전 키를 보존할 책임까지 포함해야 교체 가능한 설계가 된다.

7. 성공한 왕복 테스트 밖에서 확인해야 할 것들

암호화 후 복호화한 결과가 같다는 테스트는 필요하지만 충분하지 않다. 같은 실수가 양쪽에 들어 있어도 왕복은 성공할 수 있다. 보호 목표를 확인하려면 **어떤 조건에서 반드시 실패해야 하는지**를 테스트해야 한다.

암호화 단위 테스트에서는 nonce, 암호문, 태그를 각각 한 비트 바꿔 거절되는지 확인한다. 일기 ID나 사용자 ID를 바꾸고 같은 암호문을 복호화하는 경우도 실패해야 한다. 정상 태그를 갖지만 UTF-8이 잘못된 데이터, 공백 원문, 301 code point 원문도 따로 검사한다. 태그 검증과 서비스 입력 규칙이 서로 다른 검사임을 확인하는 사례다.

HMAC 테스트는 같은 입력의 재현성, 다른 일기나 사용자에 대한 차이, 활성 키 변경 뒤 기존 지문의 재현을 확인한다. 별도 구현으로 계산한 고정 기대값과 비교해, 생성과 비교 코드가 같은 실수를 공유할 가능성도 줄인다. DB 통합 테스트에서는 실제 저장된 원문 열이 암호문인지, 지문 열이 중간 SHA 대신 보호된 값인지 확인한다. 타인의 손상된 일기는 복호화 오류보다 권한 오류가 먼저 나야 하고, 재요청 중 실패하면 상태 변경도 롤백되어야 한다.

로그는 별도 유출 경로다. 요청과 응답 객체의 toString()에서 원문을 가려도 log.info("{}", request.sourceText())처럼 필드를 직접 출력하면 노출된다. JSON 직렬화도 toString()과 다른 경로다. 응답과 AI 요청에 필요한 원문 전달은 유지하면서, 실제 로거와 인코더를 사용하는 테스트로 가림 처리의 적용 범위를 확인했다. 예외 메시지에 원문과 키, 암호문을 넣지 않는 규칙도 함께 적용했다.

키 설정 역시 길이만 맞으면 충분하지 않다. 키 ID 문법과 Base64, 32바이트 길이를 검사해도, 같은 ID에 엉뚱한 32바이트 키를 넣은 실수는 잡지 못한다. 그래서 기존에 만들어 둔 알려진 문장의 암호문과 HMAC 검증 자료를 이용해 해당 키가 맞는지 확인한다. 잘못된 키로 검증 자료까지 새로 만들면 예전 데이터 복원 가능성을 확인할 수 없다는 점도 운영 문서에 남겼다.

기존 데이터 전환은 신규 저장과 분리했다. #442는 새 원문을 암호화하면서 기존 평문과 SHA를 읽는 호환 경로를 남겼다. 후속 #450은 원문과 버전, 지문을 함께 전환하는 별도 배치와 평문 호환 경로 제거를 다룬다. 과거 덤프와 백업에 남은 평문이나 SHA는 새 키를 사용하거나 구키를 폐기한다고 사라지지 않는다. 코드의 전환 기능과 실제 전환, 복원, 폐기 작업의 완료 여부를 따로 기록해야 한다.

다른 서비스에 적용할 때도 먼저 원문 복원이 필요한 경로와 DB 밖에 둘 키를 정할 수 있다. 이후 원문에서 파생한 지문과 로그, 구키가 필요한 백업까지 따라가면 암호화 함수 밖의 요구를 찾을 수 있다. 성능은 원문 상한과 처리량, 실제 실행 환경을 기준으로 측정해야 한다. 이 글에서는 벤치마크를 수행하지 않았으며, “암호화는 절대 병목이 되지 않는다”는 결론을 내리지 않는다.

구현과 참고 자료

- 설계 결정 ADR: 보호 경계, 알고리즘, 키 관리 대안과 선택 이유.
- 암호화와 HMAC 구현, 암호화 테스트, 지문 테스트.
- 규격과 API 동작의 근거: NIST SP 800-38D, Java 21 Cipher, RFC 2104, RFC 4648.